Implementierungsdokumentation

Projektname: IT-Quizzle

Autor: Jonas, Delwan, Leo, Robin, Triumph

Datum: 24. Januar 2026

Versionshistorie

Version	Datum	Änderungen
0.1	24.01.2026	Erste Erstellung
1.0	25.01.2026	Vollendete Version

Inhaltsverzeichnis

Inhalt

Versionshistorie 2
1.1 Anwendungsbereiche 6
1.2 Zielgruppen 6
1.3 Betriebsbedingungen 7
2. Produktumgebung 7
2.1 Software 7
2.2 Hardware 8
3. Produktfunktionen 8
F1 – Benutzeroberfläche: Game Loop 8
F2 – Login 8
F3 – Registrierung 8
F4 – Spielerstatistik anzeigen 9
4. Benutzeroberfläche 10
4.1 Lobby-Benutzeroberfläche 10
4.2 Quiz-Benutzeroberfläche 11
5. Qualitätszielbestimmungen 12
5.1 Performance 12
5.2 Zuverlässigkeit 12
5.3 Usability (Benutzerfreundlichkeit) 13
Implementierung 13
/F1/ Benutzeroberfläche: Game Loop 14
/F1.f1/ Quizfragen Oberfläche erstellen 14
/F1.f2/ Frontend: X-Anzahl an Antwortmöglichkeiten anzeigen 14
/F1.f3/ Frontend: Mirror Klassen für die Answers & Question Tabellen der Datenbank 15
/F1.f4/ Frontend: Game-Logik und Display einer Runde 15
/F1.f5/ Frontend: MatchTracking 15
/F1.f6/ Frontend: Rundenanzahlauswahl 15

/F1.b1/ Backend: GetRequest Login 16
/F1.b2/ Backend: GetRequest Answers 17
/F1.b3/ Backend: GetRequest Questions 18
/F1.b4/ Backend: GetRequest Nutzerantwortstatistik 19
/F1.b5/ Backend: GetRequest MatchHistory 19
/F1.b6/ Backend: Abfragen der Schnittstelle "Anzahl der eingetragenen Fragen" erstellen 20
/F2/ Login 21
/F2.f1/ Login im Frontend einbinden 21
/F3/ Registrierung 21
/F3.f1/ Frontend: Registrieren Oberfläche erstellen und mit Backend kommunizieren 21
/F3.b1/ Backend Schnittstelle Registrieren 22
/F4/ Spieler Statistik Anzeigen 23
/F4.f1/ Frontend: Statistiken des aktuellen Matches anzeigen und Senden 23
/F4.b1/ Backend: Post von einem Match und dessen Statistiken Hinzufügen 23
Fazit 24
Persönliche Erklärung 25

Abbildungsverzeichnis

Abbildung 1 UML-Anwendungsfalldiagramm des Quizsystems mit Player-, Admin- und Server-Interaktionen 6
Abbildung 2 Erweitertes Entity-Relationship-Diagramm (EER) des Datenbankschemas für das Quizsystem 9
Abbildung 3 Login Seite Quizzle 10
Abbildung 4 Registrierungsseite Quizzle 11
Abbildung 5 Abbildung 4 Quizzle App: Quiz Layout 12
Abbildung 6 ER-Diagramm der Datenbank mit den Tabellen für Nutzer, Fragen, Antworten, Matches und Match-Statistiken sowie möglichen Erweiterungen durch Topics und Bilder. 13
Abbildung 7 UML-Klassendiagramm der Anwendung mit Darstellung der WPF-GUI (Login, Registrierung, Spiel- und Statistikansichten), der PHP-Serverlogik sowie der zentralen Klassen für Fragen, Antworten und Match-Verwaltung. 14

Einleitung

Diese Implementierungsdokumentation beschreibt die Anforderungen und Ziele für das Quizzle Objekt und dessen Umsetzung.

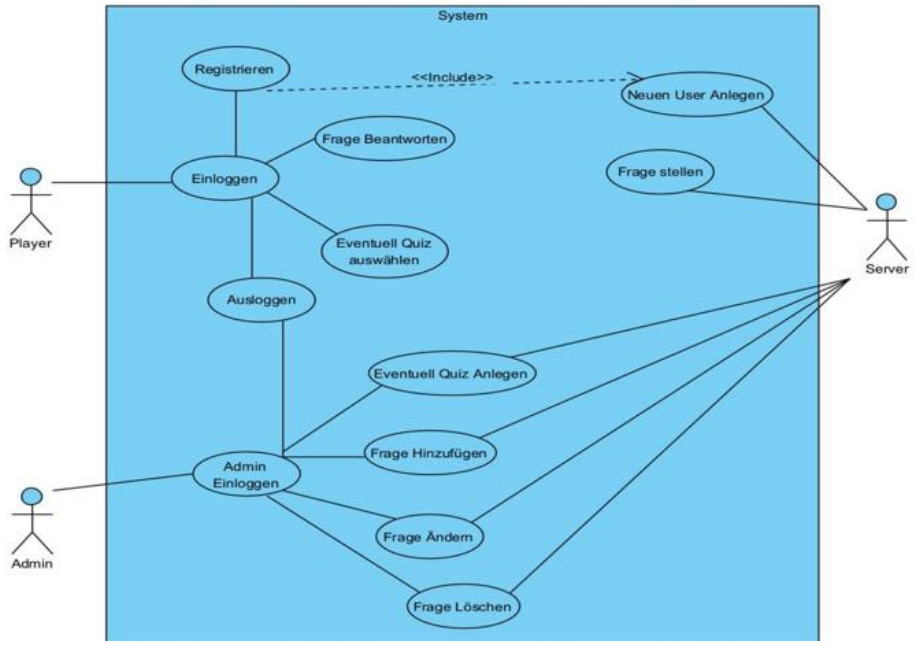

Abbildung 1 UML-Anwendungsfalldiagramm des Quizsystems mit Player-, Admin- und Server-Interaktionen

1.1 Anwendungsbereiche

- Das Produkt wird hauptsächlich im schulischen Umfeld eingesetzt, insbesondere während des Unterrichts am bib International College.
- Darüber hinaus kann das Spiel auch außerhalb der Schule, z. B. unterwegs im Zug oder zu Hause, genutzt werden, um Wissen auf spielerische Weise aufzufrischen.

1.2 Zielgruppen

- Die Hauptzielgruppe sind Schülerinnen und Schüler im Alter von 12 bis 21 Jahren.

- Das Spiel richtet sich an Lernende, die ihr Wissen spielerisch testen und erweitern möchten.
- Um das Spiel nutzen zu können, benötigen die Spieler lediglich Grundkenntnisse im Umgang mit Kahoot-ähnlichen Systemen.
- Es sind keine speziellen technischen Vorkenntnisse erforderlich.

1.3 Betriebsbedingungen

- Der Server muss eine Datenbank mit Quizfragen enthalten.
- Der Server kann auf jedem handelsüblichen Computer betrieben werden.
- In der vorgesehenen Umgebung wird der Server auf dem Laptop eines Teammitglieds ausgeführt.
- Das Spiel ist daher nur während der Unterrichtszeit verfügbar.
- Das System soll eigenständig und ohne manuelle Überwachung lauffähig sein.
- Alle Spieler benötigen eigene Laptops oder Endgeräte, um am Quiz teilnehmen zu können.

2. Produktumgebung

2.1 Software

- Das System wird auf einem Windows-Betriebssystem ausgeführt.
- Der Server verwendet eine SQL-Datenbank zur Speicherung der Quizfragen, Antworten und Spielergebnisse.
- Die Benutzeroberfläche wird mit Windows Presentation Foundation (WPF) realisiert, um ein interaktives und ansprechendes Display zu ermöglichen.
- Die Kommunikation zwischen Server und Clients erfolgt über einen Windows Communication Foundation (WCF)-Dienst.
- Die Softwarekomponenten müssen kompatibel miteinander sein und eine stabile Laufzeitumgebung bieten.

2.2 Hardware

- Es wird mindestens ein Computer benötigt, der als Server fungiert.
- Zusätzlich werden ein oder mehrere Computer als Clients eingesetzt, über die die Spieler am Quiz teilnehmen können.
- Alle Geräte müssen über eine Netzwerkverbindung verfügen, um über den WCF-Dienst miteinander zu kommunizieren.
- Die Hardware muss leistungsfähig genug sein, um die Datenübertragung und Spiellogik ohne spürbare Verzögerung zu verarbeiten.

3. Produktfunktionen

F1 – Benutzeroberfläche: Game Loop

Die Produktfunktion **F1** umfasst die vollständige Implementierung der zentralen Spieloberfläche des Quiz-Systems. Sie realisiert den durchgängigen Game Loop von der Darstellung der Quizfragen über die Auswahl und Auswertung der Antworten bis hin zur Runden- und Matchverwaltung. Dabei wurden die Benutzerinteraktion, die dynamische Anzeige von Antwortmöglichkeiten sowie das Match-Tracking umgesetzt. Die Kommunikation mit dem Backend zur Abfrage von Fragen, Antworten, Nutzerstatistiken und Matchhistorien ist vollständig integriert.

F2 – Login

Die Produktfunktion **F2** realisiert die Authentifizierung von Nutzern innerhalb des Quiz-Systems. Die Login-Oberfläche wurde im Frontend implementiert und an das Backend angebunden, um Benutzeranmeldungen über eine definierte Schnittstelle zu validieren. Dadurch wird ein sicherer Zugriff auf spiel- und nutzerspezifische Funktionen gewährleistet.

F3 – Registrierung

Die Produktfunktion **F3** ermöglicht die Neuregistrierung von Nutzern im System. Hierfür wurde eine eigenständige Registrierungsoberfläche im Frontend entwickelt, die mit der entsprechenden Backend-Schnittstelle kommuniziert. Die eingegebenen Nutzerdaten werden validiert und dauerhaft

in der Datenbank gespeichert, sodass neue Spieler erfolgreich in das System aufgenommen werden können.

F4 – Spielerstatistik anzeigen

Die Produktfunktion **F4** stellt die Erfassung und Anzeige spielbezogener Statistiken bereit. Während eines Matches werden relevante Statistiken ermittelt, im Frontend visualisiert und anschließend an das Backend übertragen. Zusätzlich können aggregierte Statistiken über alle bisher gespielten Matches eines Nutzers abgerufen und angezeigt werden, wodurch eine langfristige Leistungsübersicht entsteht.

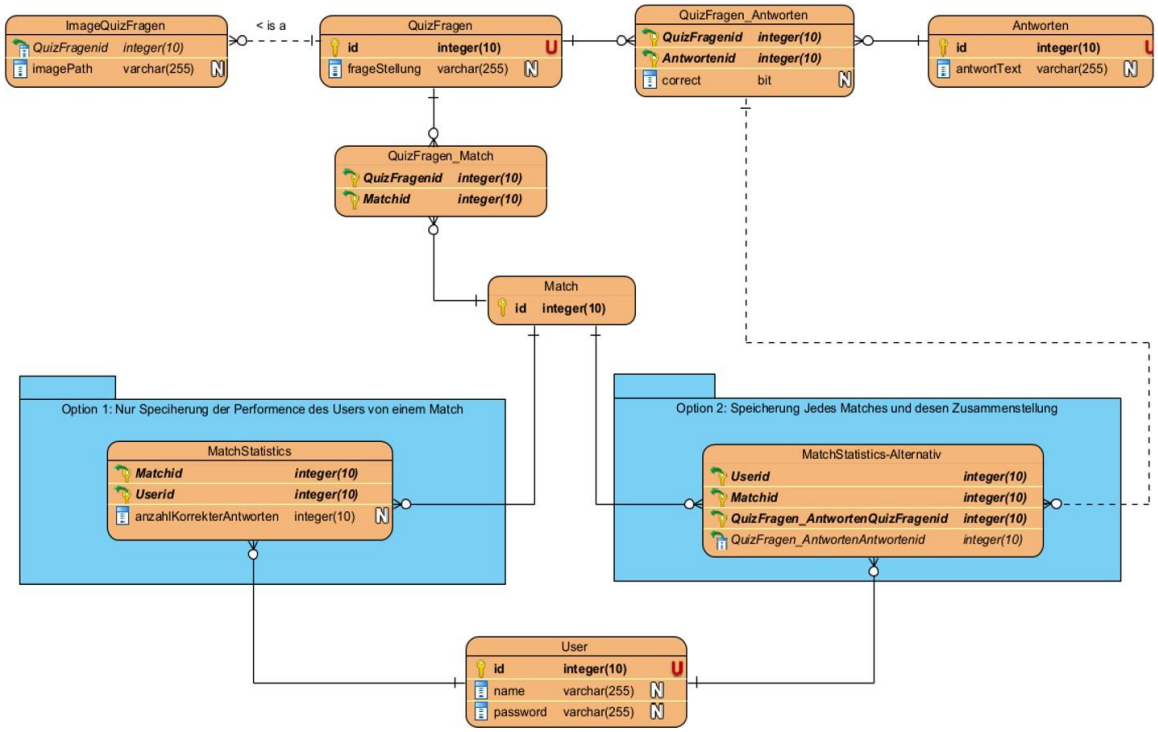

Abbildung 2 Erweitertes Entity-Relationship-Diagramm (EER) des Datenbankschemas für das Quizsystem

4. Benutzeroberfläche

4.1 Lobby-Benutzeroberfläche

- Nach erfolgreichem Login wird die Lobby-Oberfläche angezeigt.
- Oben erscheint ein Begrüßungstext mit dem Benutzernamen, z. B. „Willkommen, [Benutzername]“.
- Darunter befindet sich eine Liste der verfügbaren Spiele oder Runden mit Status und Teilnehmeranzahl.
- Buttons zum Erstellen eines neuen Spiels sowie zum Beitreten zu einem bestehenden Spiel sind vorhanden.
- Ein Chatfenster oder Nachrichtenbereich kann integriert sein, um mit anderen Spielern zu kommunizieren.
- Unten befindet sich ein Logout-Button, um sich abzumelden.

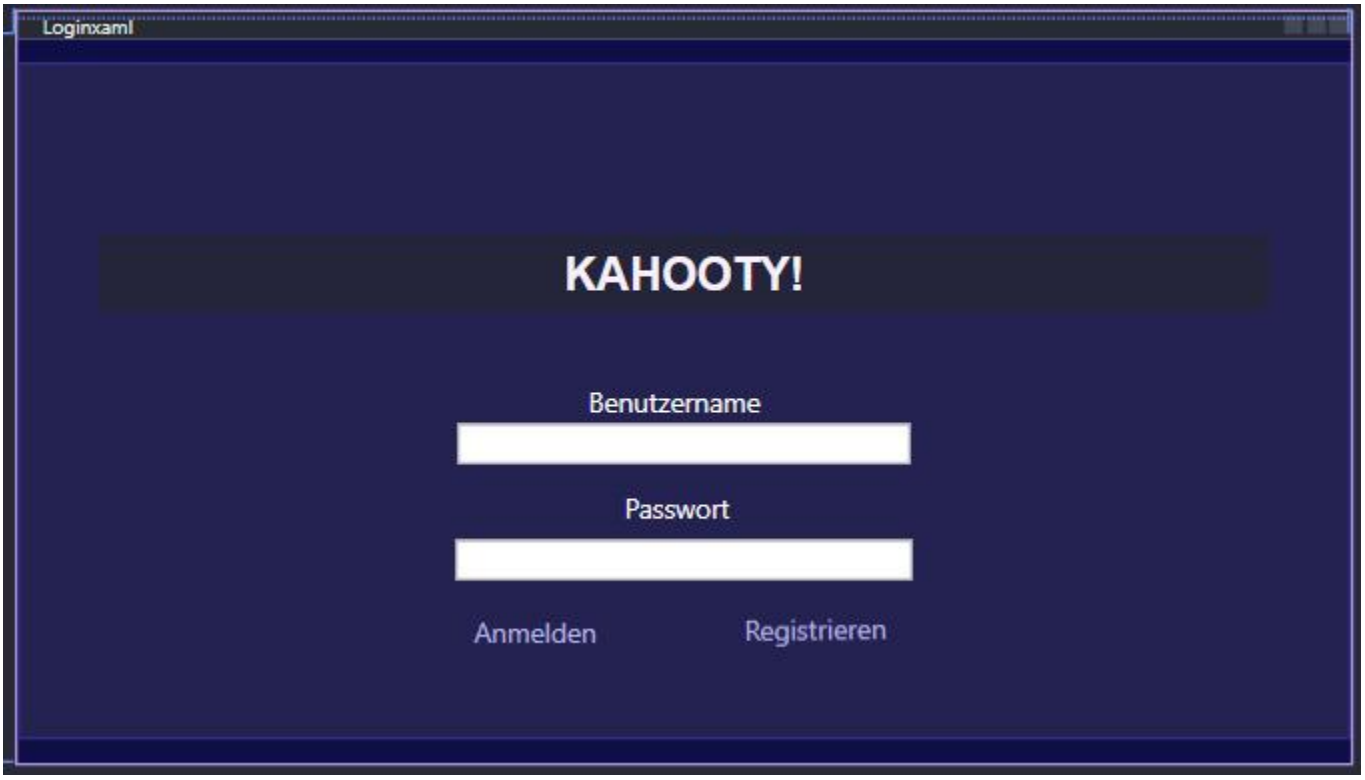

Abbildung 3 Login Seite Quizzle

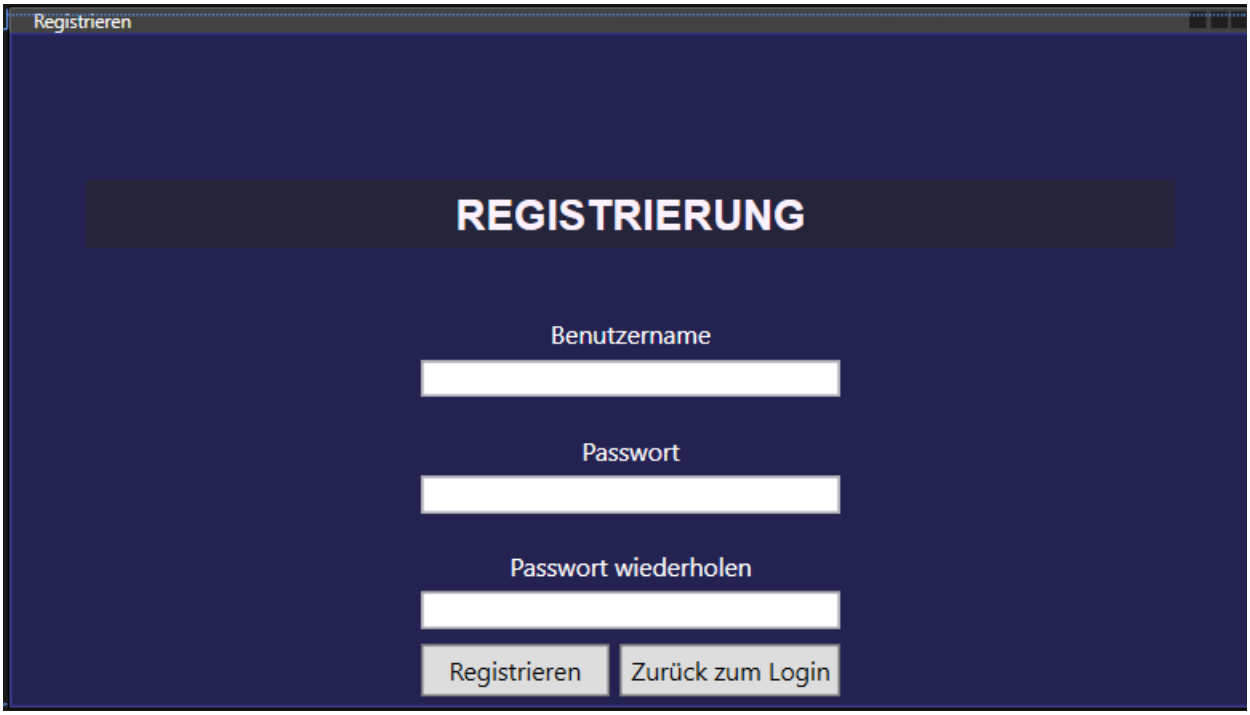

Abbildung 4 Registrierungsseite Quizzle

4.2 Quiz-Benutzeroberfläche

- Im oberen Bereich des Fensters wird die aktuelle Quizfrage als Text angezeigt.
- Im unteren Bereich erscheinen vier Schaltflächen (Buttons), die jeweils eine der möglichen Antwortoptionen enthalten.
- Die Schaltflächen sind gut sichtbar und eindeutig beschriftet, sodass die Antwortmöglichkeiten klar erkennbar und auswählbar sind.
  - Der Nutzer kann durch Klicken auf eine der vier Schaltflächen seine Antwort auswählen.
- Nach der Auswahl wird unmittelbar angezeigt, ob die Antwort richtig oder falsch war, und – sofern vorgesehen – Punkte vergeben.
- Am Ende jeder Runde erscheint in der Mitte ein Leaderboard, dass die Punkte der Spieler anzeigen.

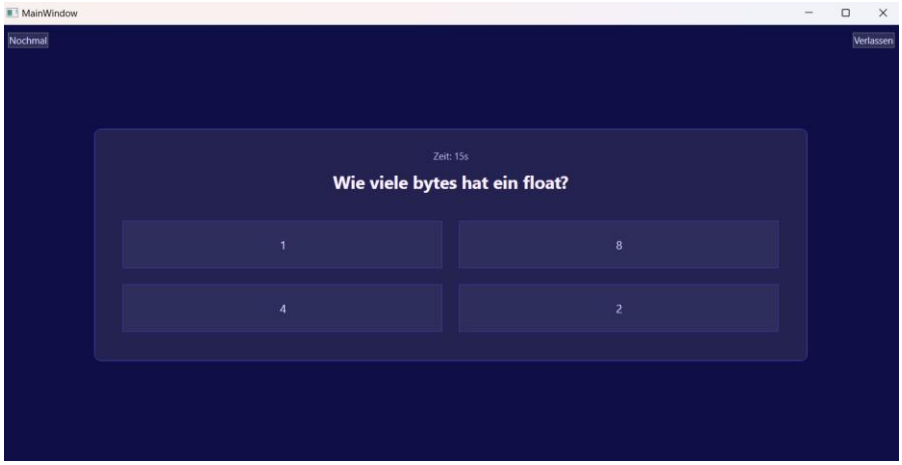

Abbildung 5 Abbildung 4 Quizzle App: Quiz Layout

5. Qualitätszielbestimmungen

5.1 Performance

- Der Server muss eine Reaktionszeit von unter 1 Sekunde auf Benutzeranfragen gewährleisten.
- Der Client (Benutzeroberfläche) muss auf Eingaben innerhalb von unter 1 Sekunde reagieren.
- Ladezeiten beim Wechsel zwischen Fragen oder Spielrunden sollen so kurz wie möglich gehalten werden, um einen flüssigen Spielablauf sicherzustellen.

5.2 Zuverlässigkeit

- Der Server und die Client-Anwendungen müssen eine hohe Verlässlichkeit aufweisen und mindestens 99 % der Zeit fehlerfrei funktionieren.
- Das System soll stabil laufen und keine oder nur sehr seltene Abstürze haben.
- Bei unerwarteten Fehlern soll das System automatisch wiederherstellbar oder neu startbar sein, ohne Datenverlust zu verursachen.

5.3 Usability (Benutzerfreundlichkeit)

- Die Anwendung muss eine intuitive Bedienung ermöglichen, sodass ein Nutzer maximal drei Klicks pro Aktion benötigt.
- Die Benutzeroberfläche (GUI) soll dem Nutzer klar und verständlich anzeigen, welche Aktionen möglich sind und was zu tun ist.
- Die Gestaltung der GUI (Farben, Layout, Beschriftungen) soll übersichtlich, konsistent und leicht verständlich sein.
- Benutzer sollen das System ohne zusätzliche Schulung bedienen können.

Implementierung

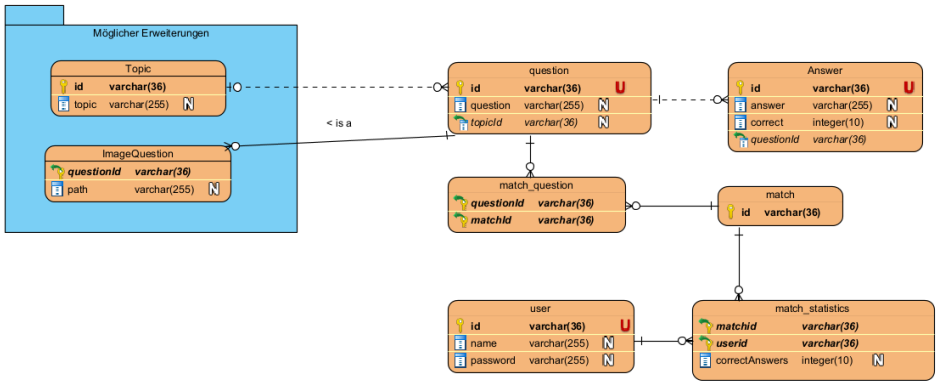

Abbildung 6 ER-Diagramm der Datenbank mit den Tabellen für Nutzer, Fragen, Antworten, Matches und Match-Statistiken sowie möglichen Erweiterungen durch Topics und Bilder.

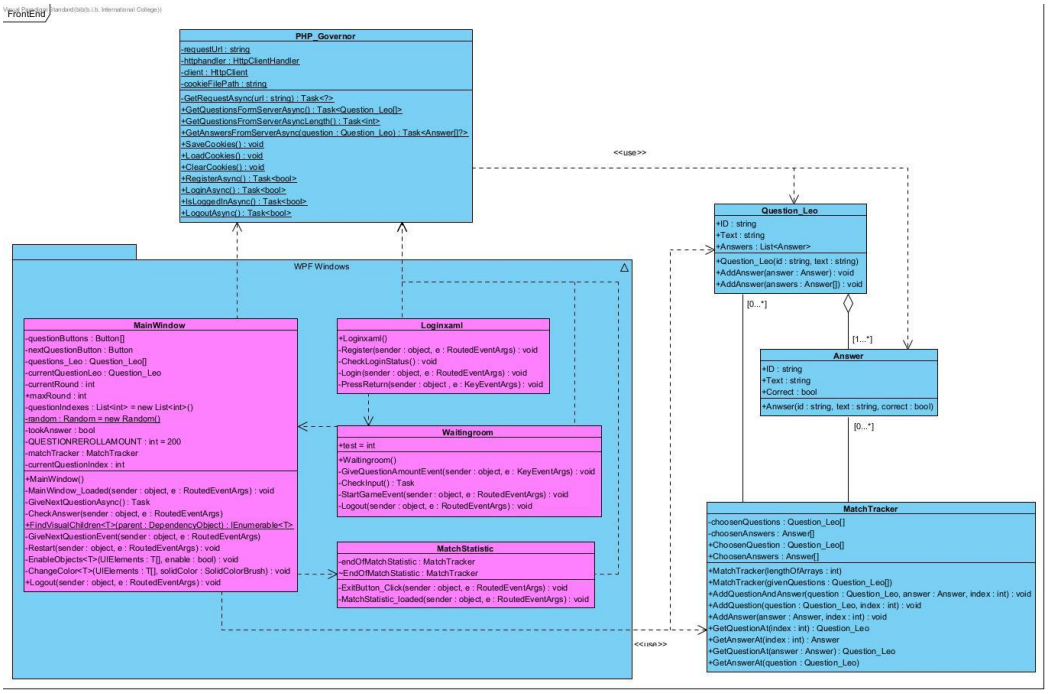

Abbildung 7 UML-Klassendiagramm der Anwendung mit Darstellung der WPF-GUI (Login, Registrierung, Spiel- und Statistikansichten), der PHP-Serverlogik sowie der zentralen Klassen für Fragen, Antworten und Match-Verwaltung.

/F1/ Benutzeroberfläche: Game Loop

/F1.f1/ Quizfragen Oberfläche erstellen

Bearbeitet von: Delwan

Die XAML-Datei definiert eine Quizfragen-Oberfläche mit einem abgedunkelten Hintergrund, einem zentrierten Fragensockel und vier Antworthilfen als Buttons im 2x2-Gitter. Jeder Button verwendet konsistente Stilvorlagen, um Layout, Farben und Interaktionen zu vereinheitlichen.

/F1.f2/ Frontend: X-Anzahl an Antwortmöglichkeiten anzeigen

Bearbeitet von: Delwan

Der Code überprüft, wie viele Antwortmöglichkeiten zu einer Frage gehören. Da es nur Fragen mit zwei oder vier Antworten gibt, wird anhand dieser Information entschieden, ob die unteren Buttons deaktiviert werden, falls die aktuelle Frage lediglich zwei Antwortoptionen enthält.

/F1.f3/ Frontend: Mirror Klassen für die Answers & Question Tabellen der Datenbank

Bearbeitet von: L.K. Zacharias, Delwan

Im Client wurden die Klassen Answer & Question_Leo erstellt, die die Tabellen Answer und Question aus der Datenbank des Servers für die C# Umgebung nutzbar machen sollen. Zusätzlich wurde die statische Klasse PHP_ Gouverneur erstellt, welche als Kommunikationsschnittstelle zwischen der Rest-API und dem Client vermittelt und anfragen stellt und in nutzbare daten konvertiert. Insgesamt war die Aufgabe mehr eine Lernerfahrung und ein Machbarkeitsnachweis, da uns es im Unterricht nicht nahe gemacht und auch kaum Hilfestellung geboten wurden.

Die Mirror-Klassen im Frontend wurden ursprünglich von mir, Delwan erstellt, später jedoch verworfen. Schlussendlich kamen die von Leo entwickelten Klassen zum Einsatz.

/F1.f4/ Frontend: Game-Logik und Display einer Runde

Bearbeitet von: Delwan

Zu Beginn des Spiels werden, basierend auf der im Warteraum angegebenen Zahl, die entsprechende Anzahl an Fragen aus der Datenbank abgerufen. Anschließend wird eine Frage ausgewählt und im Spiel angezeigt. Nach der Auswahl einer Antwort wird angezeigt, welche Antwort richtig und welche falsch war. Sobald die letzte Frage beantwortet ist, schließt sich das Spielfenster automatisch und der Spieler kehrt in den Warteraum zurück.

/F1.f5/ Frontend: MatchTracking

Bearbeitet von Robin

Während der Spieler das Spiel spielt, wird im Hintergrund durch die Match-tracking Klasse gespeichert, welche Fragen dem Spieler gestellt wurde und welche Antwort er darauf geantwortet hat. Dies wird, in Arrays gespeichert und diese daten anschließend zum Server zu schicken und anzuzeigen

/F1.f6/ Frontend: Rundenanzahlauswahl

Bearbeitet von: Delwan

Bevor das Spiel startet, gelangt man in den Warteraum. Dort kann der Spieler festlegen, wie viele Fragen das Quiz enthalten soll – jedoch nur bis zur maximalen Anzahl, die in der Datenbank verfügbar ist. Nach der Auswahl wird automatisch zum eigentlichen Spiel weitergeleitet.

/F1.b1/ Backend: GetRequest Login

Bearbeitet von Triumph

Bericht: UserController erstellt und drei Login-Methoden implementiert: eine zum Einloggen des Nutzers, eine zur Überprüfung des Login-Status und eine zum Ausloggen. Die Benutzerdaten werden serverseitig in der Session gespeichert, sodass der Anmeldezustand verwaltet werden kann.

Implementierte Schnittstelle:

Attribut	**Wert / *Beispiele (kursiv)***	**Beschreibung**
Endpoint	user	Name des Endpoints
Method	GET	http-Methode
Alias	*loginUser*	Name der aufzurufenden Methode
username	*"brotmeister"*	Benutzername des Benutzers, der sich anmelden will.
pw	*"12345"*	Passwort des Benutzers, der sich anmelden will
Rückgabe: Nachricht (als JSON-Objekt)		
isLogin	false	War der Login erfolgreich true = ja, false = nein
errorMsg	"Benutzername und Passwort stimmen nicht überein"	Fehlermeldung (nur bei true)

/F1.b2/ Backend: GetRequest Answers

Bearbeitet von: L.K. Zacharias, Delwan

Bericht Leo: Erstellung der Schnittstelle und sofortiger Funktionstest mit dem Frontend Produkt. Problemloser Arbeitsablauf, somit Aufgabe für beendet erklärt. Nutzen dieser Schnittstelle ist es alle Antworten die zu der mitgeschickten Frage aus der Datenbank an den Client zu schicken, damit der Client die Antworten in dessen Gameloop verwenden kann.

Die Klasse steht in Zusammenhang mit der **Question**-Klasse und dient dazu, die passenden Antworten bereitzustellen. Dabei können entweder alle vorhandenen Antworten oder nur die Antworten einer bestimmten Frage abgerufen werden.

Implementierte Schnittstelle:

Attribut	**Wert / *Beispiele (kursiv)***	**Beschreibung**
Endpoint	answer	Name des Endpoints
Method	GET	http-Methode
Alias	getAnswerByQuestion	
questionId	*"a10b8c84-3715-11eb-add7-2c4d544f8fe0"*	Eingabe der ID einer Frage um die zugehörigen Antwortmöglichkeiten zu bekommem
Rückgabe: Liste der Projekte (Liste von JSON-Objekten)		
id	*"a10b8c84-3715-11eb-add7-2c4d544f8fe0"*	Id der Antwort
question	*"Hier steht die Antwortmöglichkeit als Satz"*	Der String ist die representation der *Antwortmöglichkeit* .

correct	*"0"*	Angabe ob die *Antwortmöglichkeit richtig {"1"} oder falsch {"0"} ist.*

/F1.b3/ Backend: GetRequest Questions

Bearbeitet von: L.K Zacharias Delwan

Bericht Leo: Erstellung der Schnittstelle und testen dieser. Herauszufinden wie die Schnittstelle getestet werden kann, dauerte länger, als diese zu erstellen. Nutzen der Schnittstelle ist es ein neues Match pro Aufruf der Schnittstelle zu erstellen und diese mit der Anfrage dazu gesendeten Fragen zu verbinden. Am Ende wird bei Erfolg die ID des neu erstellten Matchs zurück an den Client geschickt

Bericht Delwan: Die Klasse wird benötigt, um die Fragen im Hauptspiel anzuzeigen. Innerhalb der Klasse kann wahlweise eine bestimmte Frage oder die gesamte Sammlung aller Fragen abgerufen werden.

Implementierte Schnittstelle:

Attribut	**Wert / *Beispiele (kursiv)***	**Beschreibung**
Endpoint	question	Name des Endpoints
Method	GET	http-Methode
Rückgabe: Liste der Projekte (Liste von JSON-Objekten)		
id	*"a10b8c84-3715-11eb-add7-2c4d544f8fe0"*	ID der Frage
question	*"Hier steht die Frage als Satz"*	Der String ist die Repräsentation der Frage.

/F1.b4/ Backend: GetRequest Nutzerantwortstatistik

Bearbeitet von Robin

Man kann dem Server nun nach seiner Antwortstatistik Fragen, die anfrage wird durch den Match_StatisticController angenommen und anschließend von dem Match_StatisticModel durchgeführt

Implementierte Schnittstelle:

Attribut	**Wert / *Beispiele (kursiv)***	**Beschreibung**
Endpoint	match_statistic	Name des Endpoints
Method	GET	http-Methode
Alias	getMatchStatsByUser	Die UserId wird in $_SESSION['userId'] gespeichert
Rückgabe: Liste der Projekte (Liste von JSON-Objekten)		
matchId	*"a10b8c84-3715-11eb-add7-2c4d544f8fe0"*	Id des Matches an dem der Nutzer teilgenommen hat
correctAnswers	*"2"*	Anzahl der im Match vom Nutzer richtig beantworteten Fragen
questionAmount	*"5"*	Anzahl der im Match gestellten Fragen

/F1.b5/ Backend: GetRequest MatchHistory

Bearbeitet von Jonas Aldejohann

Es wurde ein GET-Endpoint zur Abfrage der MatchHistory eines Matches umgesetzt. Der Endpoint erwartet eine matchId und wird vom Match_questionController verarbeitet. Der Controller ruft eine Model-Methode auf, die die Datenbankabfrage übernimmt. Im Model werden die zugehörigen Fragen eines Matches über einen SQL-LEFT-JOIN geladen. Zurückgegeben werden matchId, questionId und der

Fragetext. Die Ergebnisse werden als JSON-Liste ausgegeben, bei leeren Daten mit einer Hinweisnachricht.

Implementierte Schnittstelle:

Attribut	**Wert / *Beispiele (kursiv)***	**Beschreibung**
Endpoint	match_question	Name des Endpoints
Method	GET	http-Methode
Alias	getMatchHistoryByID	
matchId	*"a10b8c84-3715-11eb-add7-2c4d544f8fe0"*	ID des Matches
Rückgabe: Liste der Projekte (Liste von JSON-Objekten)		
questionId	*"a10b8c84-3715-11eb-add7-2c4d544f8fe0"*	Id der Question
question	*"Hier steht die Frage als Satz"*	Der String ist die representation der Frage.

/F1.b6/ Backend: Abfragen der Schnittstelle "Anzahl der eingetragenen Fragen" erstellen

Bearbeitet von Jonas Aldejohann

Es wurde ein GET-Endpoint question mit dem Alias getCount implementiert.
Der Endpoint dient zur Abfrage der Gesamtanzahl der in der Datenbank gespeicherten Fragen. Der Controller ruft dafür die Methode selectQuestionCount() im QuestionModel auf. Im Model wird mittels SQL COUNT (*) die Anzahl der Einträge in der Tabelle question ermittelt. Das Ergebnis wird als JSON-Objekt mit dem Attribut count zurückgegeben.

Implementierte Schnittstelle:

Attribut	**Wert / *Beispiele (kursiv)***	**Beschreibung**
Endpoint	question	Name des Endpoints
Method	GET	http-Methode
Alias	getCount	
Rückgabe: Liste der Projekte (Liste von JSON-Objekten)		
count	*"10"*	Anzahl der in der Datenbank gespeicherten Fragen

/F2/ Login

/F2.f1/ Login im Frontend einbinden

Bearbeitet von Triumph

Bericht: Ich habe die Login-Funktion aus dem Backend ins Frontend geholt und nutze dafür API-Anfragen. Dazu gehören Anmelden, Status prüfen und Abmelden. Außerdem werden Cookies gesetzt, damit der Nutzer auch nach Neustart des Programms drinbleibt. Beim Start lädt die App die Cookies wieder, damit eine vorhandene Sitzung automatisch weiterläuft. Die GUI habe ich für die Anmeldung auch erweitert.

/F3/ Registrierung

/F3.f1/ Frontend: Registrieren Oberfläche erstellen und mit Backend kommunizieren

Bearbeitet von Triumph, Jonas Aldejohann

Bericht: Die GUI für die Registrierung erweitert. Nutzer können jetzt direkt im Programm ein Konto erstellen. Dabei erhält der Nutzer einen Hinweis, wenn der gewählte Nutzername die maximale Länge von 8 Zeichen überschreitet.

Abkapslung der Registrierung des Logins:

Die Registrierung wurde funktional vom Login getrennt und eigenständig implementiert. Dadurch ist eine klarere Trennung der Zuständigkeiten zwischen Anmeldung und Kontoerstellung entstanden. Zusätzlich wird das Passwort bei der Registrierung nun doppelt abgefragt. Beide Eingaben werden verglichen, um Fehleingaben zu vermeiden. Nur bei übereinstimmenden Passwörtern wird die Registrierung erfolgreich durchgeführt.

/F3.b1/ Backend Schnittstelle Registrieren

Bearbeitet von Triumph

Es wurde eine Registrierungsfunktion implementiert, über die neue Nutzerkonten erstellt werden können. Die eingegebenen Daten werden in der Datenbank gespeichert. Dabei wird geprüft, ob der Nutzername bereits vergeben ist und ob er die maximale Länge von 8 Zeichen einhält.

Implementierte Schnittstelle:

Attribut	**Wert / *Beispiele (kursiv)***	**Beschreibung**
Endpoint	user	Name des Endpoints
Method	POST	http-Methode
name	*"PapemanDerEchte"*	Nickname des Nutzers
password	*"password1234"*	
Rückgabe: Message (JSON-Objekte)		

isError	false	Ist ein Fehler aufgetreten? true = ja, false = nein
msg		Fehlermeldung (nur bei true)
ex		Technische Fehlermeldung (nur bei true)

/F4/ Spieler Statistik Anzeigen

/F4.f1/ Frontend: Statistiken des aktuellen Matches anzeigen und Senden

sobald das match vorbei ist, werden die von der Match Tracker klasse gespeicherten daten an den Server geschickt und in der Datenbank gespeichert und anschließend Dem Spieler gezeigt (Das hat alles Robin und ChatGPT gemacht, weil uns legit nichts beigebracht wird)

/F4.b1/ Backend: Post von einem Match und dessen Statistiken Hinzufügen

Bearbeitet von: L.K. Zacharias

Erstellung der Schnittstelle und testen dieser. Herauszufinden wie die Schnittstelle getestet werden kann, dauerte länger, als diese zu erstellen. Nutzen der Schnittstelle ist es ein neues Match pro Aufruf der Schnittstelle zu erstellen und diese mit der Anfrage dazu gesendeten Fragen zu verbinden. Am Ende wird bei Erfolg die ID des neu erstellten Matchs zurück an den Client geschickt

Implementierte Schnittstelle:

Attribut	**Wert / *Beispiele (kursiv)***	**Beschreibung**
Endpoint	match_question	Name des Endpoints
Method	POST	http-Methode

Liste von Fragen **(Liste von JSON-Objekten)**		
questionId	*"0bb28278-d28a-11e7-b93f-2c4d544f8fe0"*	ID von Fragen, die Teil des Matchs waren.
Rückgabe: Message (JSON-Objekte)		
isError	false	Ist ein Fehler aufgetreten? true = ja, false = nein
matchId	*"A2b28278-d28a-11e7-b93f-2c4d544f8fe0"*	matchId (nur bei isError = false)
msg		Fehlermeldung (nur bei true)
ex		Technische Fehlermeldung (nur bei true)

Fazit

Das vorliegende Dokument beschreibt die vollständige Entwicklung und Implementierung eines interaktiven Quiz-Systems, das gezielt zur spielerischen Wissensfestigung im IT-Bereich bei Schülern eingesetzt wird. Durch den erfolgreichen Einsatz von WPF, WCF und SQL in der Windows-Umgebung wurde eine stabile technische Basis realisiert, die die Motivation sowie den Lernerfolg der Zielgruppe durch integrierte Gamification-Elemente nachhaltig unterstützt.

Persönliche Erklärung

Dieses Dokument wurde von Jonas, Delwan, Leo, Robin und Triumph verfasst